

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of THU 5 JUNE 2025 – Theory Part

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory syntax and semantics of languages, divided in four sections: (75% of the final grade)

- 1) regular expressions and finite automata (1 exercise × 10 points)
- 2) syntax analysis and parsing methodologies (1 exercise × 10 points)
- 3) grammar design – translation – semantic analysis (2 exercises × 5 points)
- 4) *bonus* question (1 exercise × 5 points)

Lab compiler design by Flex and Bison (25% of the final grade)

- 1) basic questions (30 points)
- 2) *bonus* question (5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Regular expressions and finite automata (10 points)

Consider the regular expression R below, over the two-letter alphabet $\Sigma = \{ a, b \}$

$$R = (a^+ \mid bb \mid bba)^*$$

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) List all the strings of length ≤ 3 (less than or equal to 3) that are generated by the regular expression R .
 - (b) Say which strings of those listed at point (a) are ambiguous (if any), and for each of them show two different derivations.
 - (c) By using the Berry-Sethi (BS) method, build a deterministic automaton A equivalent to the regular expression R .
 - (d) Determine if the automaton A of point (c) is minimal, and if necessary minimize it.
 - (e) Construct the complement automaton B of the minimal version of automaton A , and verify that automaton B rejects all the strings found at point (a), and accepts the three shortest strings invalid for automaton A . Is the automaton B minimal ? Justify your answer.
 - (f) By using the Brzozowski-McCluskey (BMC) method, find a regular expression R' equivalent to the complement automaton B .
-

Solution

- (a) Here are the seven short strings of length ≤ 3 generated by the regular expression R and sorted lexicographically (more precisely alphabetically): $\varepsilon, a, a a, a a a, b b, a b b$ and $b b a$.
- (b) The three short strings $a a, a a a$ and $b b a$ (see point (a)) are ambiguous for the regular expression R (so expression R is itself ambiguous). Here are two derivations for the ambiguous string $a a$:

$$(a^+ | b b | b b a)^{\overbrace{*=1}^{\text{choose}}} \Rightarrow \underbrace{a^+}_{\text{choose}} | b b | b b a \Rightarrow a^{\overbrace{+=2}^{\text{choose}}} \Rightarrow a a$$

otherwise (some derivation steps, for union and cross, are compacted):

$$(a^+ | b b | b b a)^{\overbrace{*=2}^{\text{choose}}} \Rightarrow \left(\underbrace{a^+}_{\text{choose}} | b b | b b a \right) \left(\underbrace{a^+}_{\text{choose}} | b b | b b a \right) \xrightarrow[\text{union}]{2 \text{ steps}} a^{\overbrace{+=1}^{\text{choose}}} a^{\overbrace{+=1}^{\text{choose}}} \xrightarrow[\text{cross}]{2 \text{ steps}} a a$$

For completeness, here are four derivations for string $a a a$:

$$(a^+ | b b | b b a)^{\overbrace{*=1}^{\text{choose}}} \Rightarrow \underbrace{a^+}_{\text{choose}} | b b | b b a \Rightarrow a^{\overbrace{+=3}^{\text{choose}}} \Rightarrow a a a$$

otherwise (some derivation steps, for union and cross, are compacted):

$$(a^+ | b b | b b a)^{\overbrace{*=2}^{\text{choose}}} \Rightarrow \left(\underbrace{a^+}_{\text{choose}} | b b | b b a \right) \left(\underbrace{a^+}_{\text{choose}} | b b | b b a \right) \xrightarrow[\text{union}]{2 \text{ steps}} a^{\overbrace{+=1}^{\text{choose}}} a^{\overbrace{+=2}^{\text{choose}}} \xrightarrow[\text{cross}]{2 \text{ steps}} a a a$$

or also:

$$(a^+ | b b | b b a)^{\overbrace{*=2}^{\text{choose}}} \Rightarrow \left(\underbrace{a^+}_{\text{choose}} | b b | b b a \right) \left(\underbrace{a^+}_{\text{choose}} | b b | b b a \right) \xrightarrow[\text{union}]{2 \text{ steps}} a^{\overbrace{+=2}^{\text{choose}}} a^{\overbrace{+=1}^{\text{choose}}} \xrightarrow[\text{cross}]{2 \text{ steps}} a a a$$

and a final alternative (any pair of these four is acceptable for the answer):

$$(a^+ | b b | b b a)^{\overbrace{*=3}^{\text{choose}}} \Rightarrow \left(\underbrace{a^+}_{\text{choose}} | b b | b b a \right) \left(\underbrace{a^+}_{\text{choose}} | b b | b b a \right) \left(\underbrace{a^+}_{\text{choose}} | b b | b b a \right) \xrightarrow[\text{union}]{3 \text{ steps}} a^{\overbrace{+=1}^{\text{choose}}} a^{\overbrace{+=1}^{\text{choose}}} a^{\overbrace{+=1}^{\text{choose}}} \xrightarrow[\text{cross}]{3 \text{ steps}} a a a$$

And here are two derivations for string $b b a$:

$$(a^+ | b b | b b a)^{\overbrace{*=1}^{\text{choose}}} \Rightarrow a^+ | b b | \underbrace{b b a}_{\text{choose}} \Rightarrow b b a$$

otherwise (some union derivation steps are compacted):

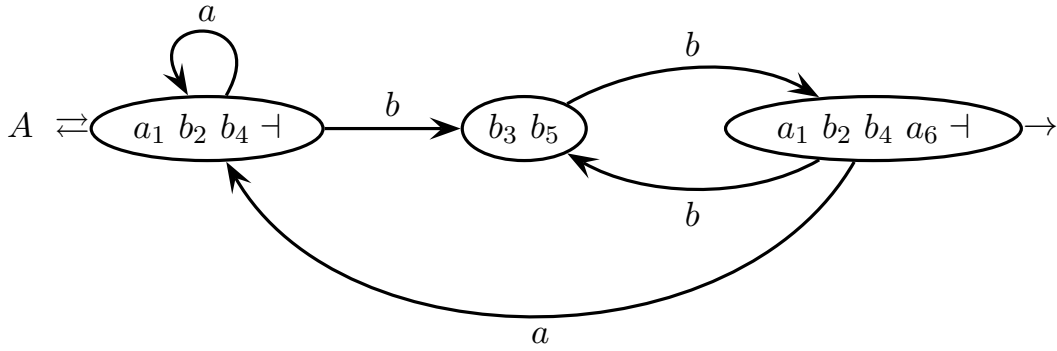
$$(a^+ | b b | b b a)^{\overbrace{*=2}^{\text{choose}}} \Rightarrow \left(a^+ | \underbrace{b b}_{\text{choose}} | b b a \right) \left(\underbrace{a^+}_{\text{choose}} | b b | b b a \right) \xrightarrow[\text{union}]{2 \text{ steps}} b b a^{\overbrace{+=1}^{\text{choose}}} \Rightarrow b b a$$

(c) Here is the Berry-Sethi method applied to the regular expression R :

$$R_{\#} = (a_1^+ \mid b_2 b_3 \mid b_4 b_5 a_6)^*$$

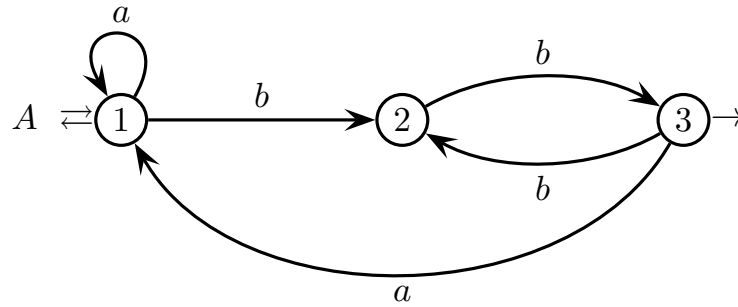
initials	$a_1 \ b_2 \ b_4 \ \neg$
terminal	followers
a_1	$a_1 \ b_2 \ b_4 \ \neg$
b_2	b_3
b_3	$a_1 \ b_2 \ b_4 \ \neg$
b_4	b_5
b_5	a_6
a_6	$a_1 \ b_2 \ b_4 \ \neg$

The resulting deterministic automaton A , equivalent to the regular expression R , is:

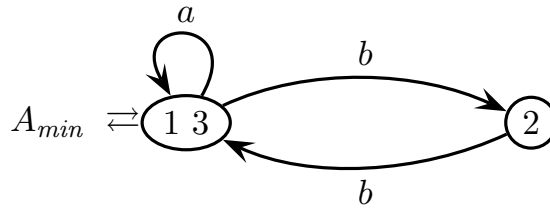


with three states.

(d) Here is the automaton A after renaming the states:

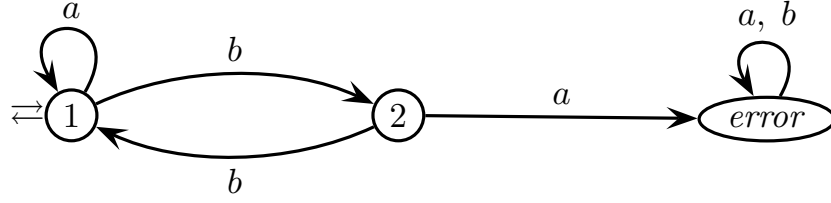


Automaton A is not minimal, since states 1 and 3 are equivalent, as both are final, go to state 1 by letter a and go to state 2 by letter b . Here is the minimal version A_{min} of automaton A :

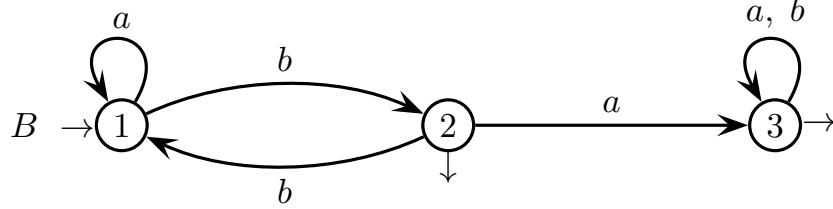


where the states 1 and 3 are merged into one state named 13, which is initial (as state 1 is so).

(e) Completion of the (minimal) automaton A_{min} with an error state (state 13 is renamed 1):



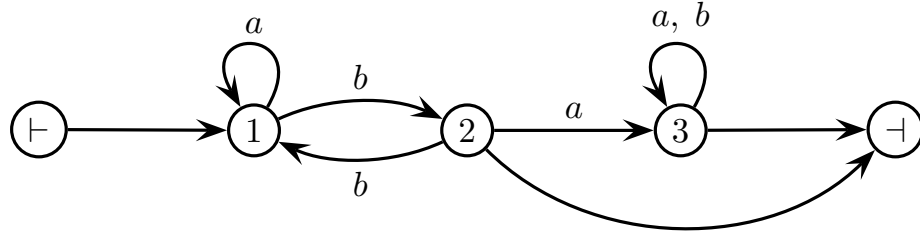
Here is the complement automaton B (with renaming the error state):



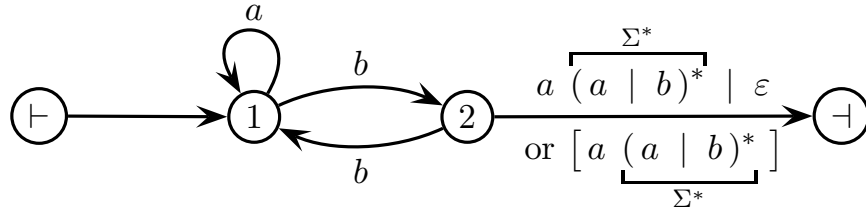
Automaton B rejects the seven strings ε , a , aa , aaa , bb , abb and bba that are valid for automaton A (as it never reaches a final state). It accepts the three shortest strings b , ab and ba that are invalid for automaton A (or for the minimal version A_{min}), as expected.

Automaton B is minimal, since the final states 2 and 3 are distinguishable by letter b .

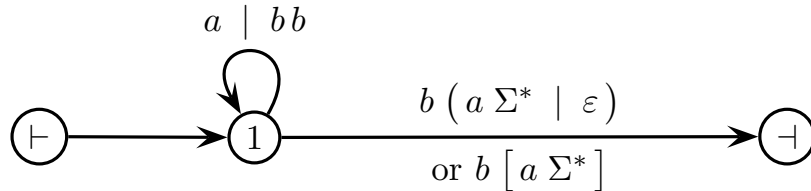
(f) Here is the BMC method applied to automaton B . First normalize automaton B :



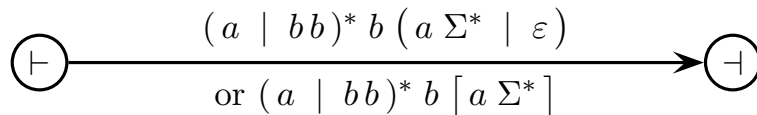
since the original initial and final states have ingoing and outgoing arcs. Eliminate node 3:



where “[]” is the optionality operator. Eliminate node 2:



Eliminate node 1:



Therefore a possible regular expression R' , complement of expression R , is:

$$R' = (a \mid bb)^* b (a \Sigma^* \mid \varepsilon) = (a \mid bb)^* (ba \Sigma^* \mid b)$$

or equivalently:

$$R' = (a \mid bb)^* b [a \Sigma^*]$$

The latter (shorter) formulation of expression R' uses the optionality operator “[]”.

If node 1 is eliminated before node 2, the following equivalent regular expression is obtained:

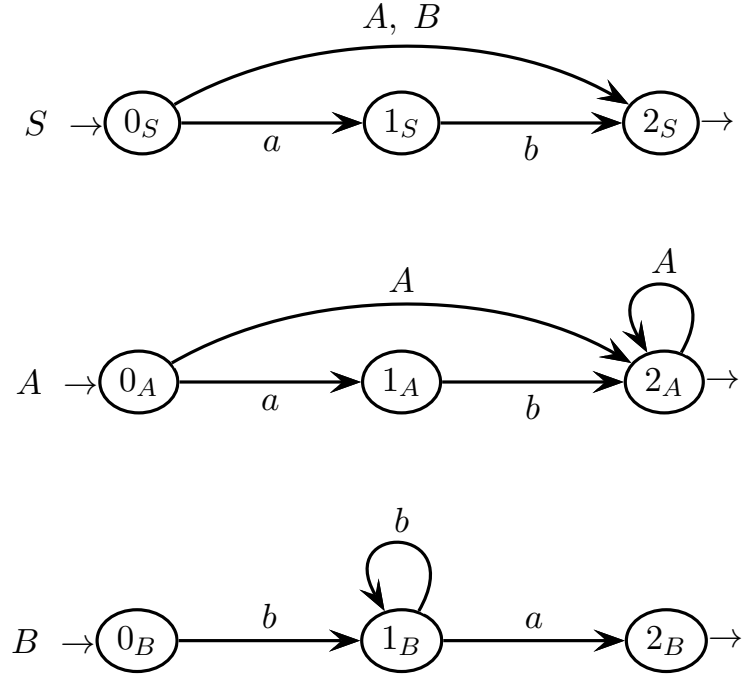
$$R' = a^* b (b a^* b)^* [a \Sigma^*]$$

For the sake of accuracy, note that expression R' generates the three strings b , ab and ba that are not generated by automaton A (see point (e)) and consequently are not generated by expression R either, as expected.

A curiosity. The regular expression R is ambiguous (see point (b)), since the union term $bb a = bb \cdot a$ is redundant, and it is easy to see that expression R can be disambiguated as $R = (a \mid bb)^*$. Therefore, from above it holds $R' = R b [a \Sigma^*]$, that is, $L(R') = L(R) L(b [a \Sigma^*])$, i.e., all the strings of the complement language $\overline{L(R)} = L(R')$ are obtained from those of the original language $L(R)$ by appending a suffix of type $b [a \Sigma^*]$, i.e., b , ba , baa , bab , etc.

2 Syntax analysis and parsing methodologies (10 points)

Consider the following grammar G , described by a machine net, over the two-letter terminal alphabet $\Sigma = \{a, b\}$ and the three-symbol nonterminal alphabet $V = \{S, A, B\}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw at least three different syntax trees of the valid string below:

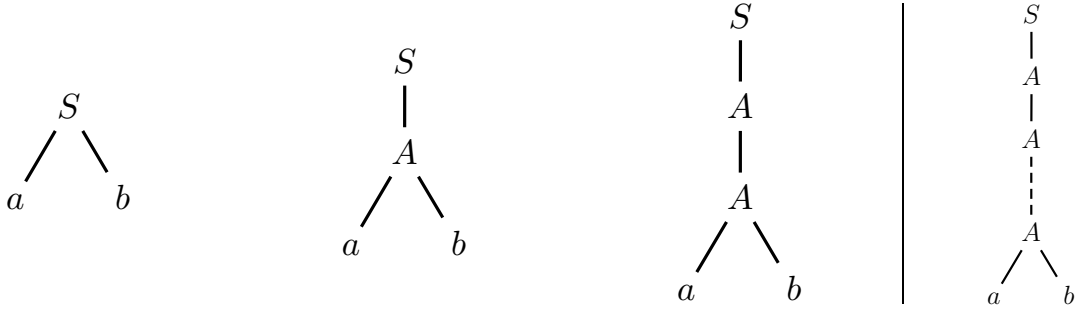
ab

How many syntax trees does this string admit?

- (b) Draw the complete ELR pilot of grammar G (part of it is already given), list all the conflicts (if any), and determine whether grammar G is of type ELR(1) or not. Explain your answer.
- (c) Find all the guide sets of the machine net of grammar G (on the call arcs and exit arrows), list all the conflicts (if any), and determine whether G is of type ELL(1) or not. Explain your answer.
- (d) Now consider the language described by machine A alone. Is the language regular or context free? Explain your answer.

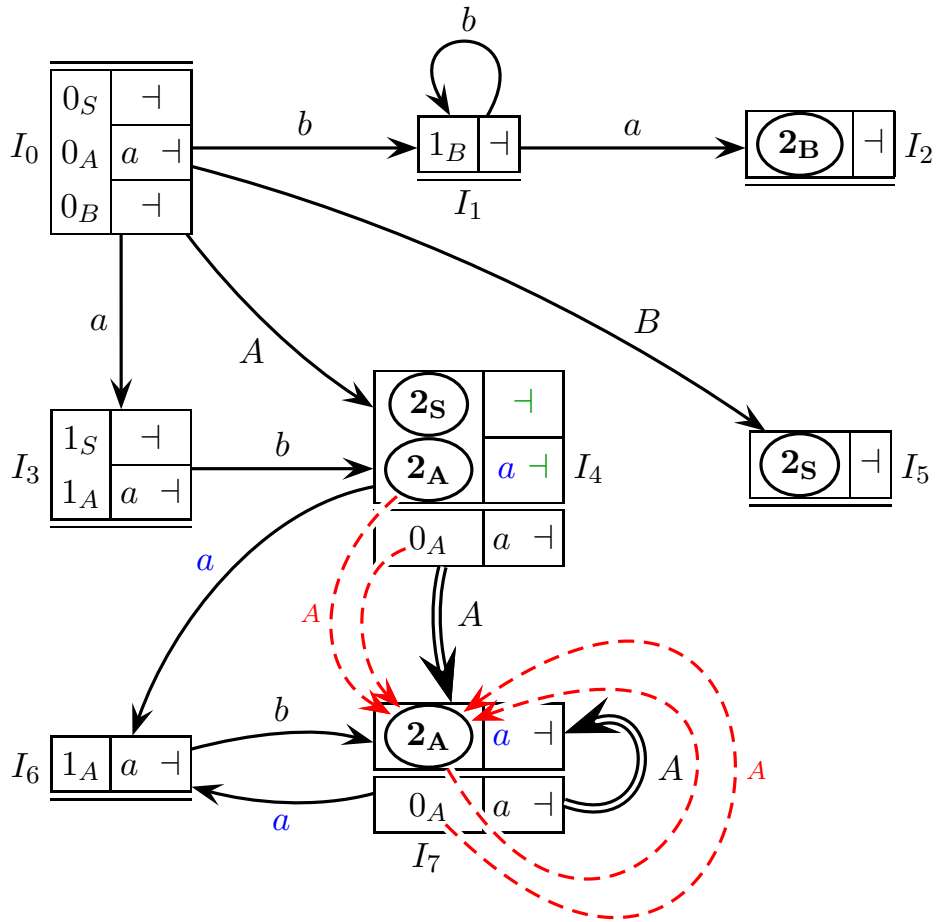
Solution

(a) Here are three different syntax trees for the valid string “ ab ” (below, left) :



Due to the recursive structure of machine A , and specifically to the arc $(0_A \xrightarrow{A} 2_A)$, there are infinitely many trees that represent string “ ab ”, all distinguished only by the number of nested rules of A (see the sketched tree above, right, with an arbitrarily long stem). Moreover, string “ ab ” can also be produced directly in S . The fact that there are multiple syntax trees for this string already suggests that grammar G cannot be ELR(1) or ELL(1).

(b) Here is the complete pilot of grammar G , with eight m-states:



There are conflicts of SR, RR and Convergence type in several m-states, namely (complete list):

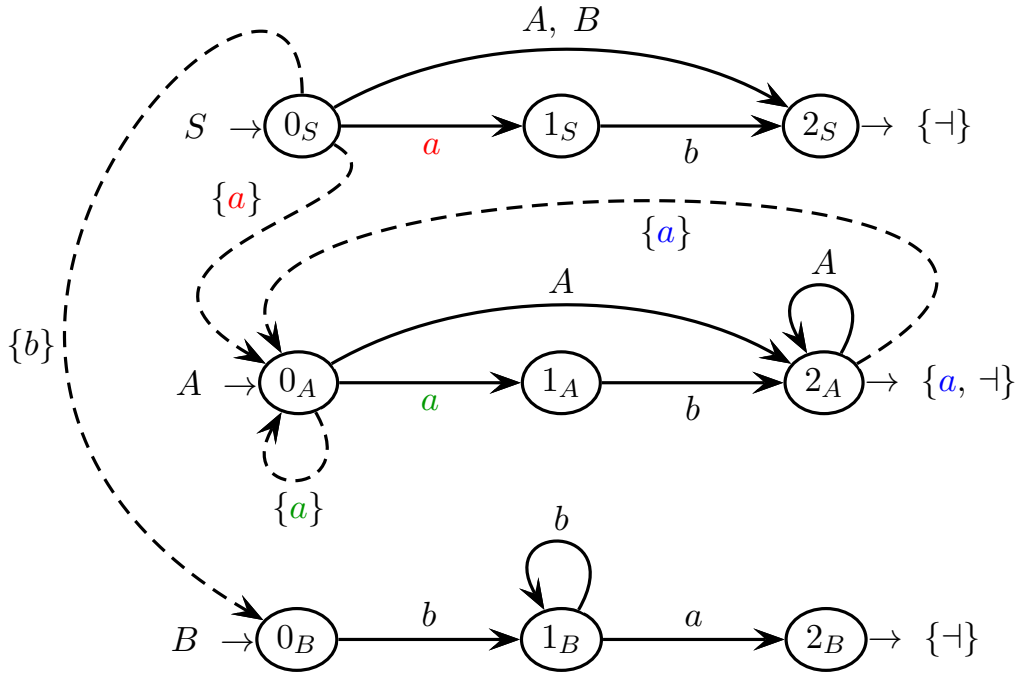
- in m-state I_4 , a shift-reduce conflict between item $\langle 2_A, a \rangle$ and outgoing edge a
- in m-state I_7 , a shift-reduce conflict between item $\langle 2_A, a \rangle$ and outgoing edge a

- in m-state I_4 , a reduce-reduce conflict between item $\langle \textcircled{2_S}, - \rangle$ and $\langle \textcircled{2_A}, - \rangle$
- a convergence conflict in m-state I_7 , because from m-state I_4 both items $\langle \textcircled{2_A}, a - \rangle$ and $\langle 0_A, a - \rangle$ shift by symbol A to item $\langle \textcircled{2_A}, a - \rangle$, and their look-ahead sets coincide
- a second convergence conflict in m-state I_7 , because from I_7 itself both items $\langle \textcircled{2_A}, a - \rangle$ and $\langle 0_A, a - \rangle$ shift by symbol A to item $\langle \textcircled{2_A}, a - \rangle$, and their look-ahead sets coincide

The shift-reduce conflicts arise from the ambiguity in state 2_A , since the recognition of a substring “ ab ” may be performed either by the current rule of nonterminal A , as $2_A \xrightarrow{A} 2_A$, or by the parent rule, provided it is still a rule of A . The reduce-reduce conflict is caused by the ambiguity of nonterminal S , since the same string “ ab ” can be recognized either directly by machine S or by a rule of A , as $0_S \xrightarrow{A} 2_S$. Finally, the convergence conflicts arise from the double recursion in the rule of A . In fact, whether we shift by A from 0_A or from 2_A , we still find ourselves in 2_A again with the same look-ahead, even though the recognition process has so far followed entirely different paths. In any case, whatever the reason for the conflicts is, the grammar G cannot be ELR(1) – it is an ambiguous grammar after all.

Note that the pilot has three more multiple-type transitions in addition to the previous two, which are convergent. Those three transitions are $I_0 \xrightarrow{a} I_3$, $I_0 \xrightarrow{A} I_4$ and $I_3 \xrightarrow{b} I_4$, all of which are double-type. Yet none of them is convergent, so they would be harmless to ELR determinism.

(c) Here are the guide sets on the call arcs and exit arrows:



Clearly, there are conflicts in the parser control flow graph as well, therefore we confirm that the net is not ELL(1). Overlapping guide sets appear between the following pairs of arcs in the graph:

- from 0_S , the transition to 1_S and the call arc to 0_A (red)
- from 0_A , the transition to 1_A and the call arc to 0_A itself (green)
- from 2_A , the exit arrow and the call arc to 0_A (blue)

Each ELL conflict can be traced back to an ambiguity in the machines, like the ELR conflicts.

- (d) Answering this question requires a minimum of reasoning. First of all, we notice that all the calls in machine A to other machines are recursive to A itself. Moreover, the only way machine A can produce a string of terminal characters is through the arcs from 0_A to 1_A and then to 2_A . This path is forced and always produces a substring “ ab ”. Nothing else in machine A can produce a terminal, so the language recognized by A must be a subset of $(ab)^*$. We can immediately see that the empty string is not recognized, because 0_A is not final. We can also see that all the other strings that consist of a repetition of substring “ ab ” are recognized. In fact, we know (from the answer to question (a)) that machine A recognizes a single substring “ ab ”. Thus we can simply append any number of substrings “ ab ” after the first one, by taking the recursive call of A from the self-loop on 2_A any number of times we want, and each recursive call will recognize a substring “ ab ”. Therefore we have $L(A) = (ab)^+$, clearly a regular language.

A curiosity. Note that the entire language $L(G) = L(S)$ is regular, not only the language $L(A)$. Clearly $L(B) = b^+a$, as said $L(A) = (ab)^+$, and finally $L(S) = ab \mid A \mid B = ab \mid (ab)^+ \mid b^+a = (ab)^+ \mid b^+a$, which is a regular language. Therefore, it is surely possible to find a deterministic grammar G' (of ELR and even ELL type) equivalent to grammar G .

3 Grammar design – translation – semantic analysis (5 + 5 points)

1. Consider the following code fragment in (a slight variation of) the Rulebook language, which describes an implementation of the “tic-tac-toe” game (the comments `//` are not part of the language):

```
coroutine play ( ) -> TicTacToe {                               // declaration of coroutine
    frame board : Board
    while ! board.is_full ( ) {
        action mark (Int x, Int y) [                            // interruption / resumption point
            x < 3, x >= 0, y < 3, y >= 0, board.get (x, y) == 0    // constraints
        ] { board.set (x, y) }                                    // end of the int. / res. point
        if board.three_in_a_line ( ) { return }
        board.change_player ( )
    }
}
```

The Rulebook language allows to declare coroutines, which are functions that can be interrupted and resumed, by prefixing them with the `coroutine` keyword, e.g., the `play` coroutine declared above.

Within a coroutine, the `action` keyword is used to declare an interruption / resumption point, where new values, declared as parameters of the interruption point, can be obtained from the caller, e.g., the `mark` interruption / resumption point within the `play` coroutine above. Such parameters are subjected to constraints (see the example above), expressed as a (possibly empty) list of expressions. For instance, here is a caller function `main` that repeatedly resumes the coroutine `play` at the `mark` point and each time passes parameters:

```
function main ( ) -> Int {
    let game = play ( )
    game.mark (0, 0)      // resume mark and pass parameters 0, 0
    game.mark (1, 0)      // resume mark and pass parameters 1, 0
    game.mark (1, 1)      // ...
    game.mark (2, 0)      // ...
    game.mark (2, 2)      // ...
    return game.board.three_in_a_line ( )
}
```

Assume that expressions and types are modeled by the nonterminals `EXPR` and `TYPE`, respectively, and that the identifiers are modeled by the terminal `ID`; none of them needs to be expanded. Furthermore, the declaration of a coroutine is modeled by the following grammar fragment:

$$\left\{ \begin{array}{l} \langle \text{COROUTINE} \rangle \rightarrow \text{coroutine ID} \text{ “(” } \langle \text{PARAM_LST} \rangle \text{ “)” “->” } \langle \text{TYPE} \rangle \text{ “{” } \langle \text{BLOCK} \rangle \text{ “} \} \text{”} \\ \langle \text{BLOCK} \rangle \rightarrow \langle \text{STATEMENT} \rangle^+ \\ \langle \text{STATEMENT} \rangle \rightarrow \langle \text{VAR_DCL} \rangle \mid \langle \text{INTERRUPT} \rangle \mid \langle \text{WHILE} \rangle \mid \langle \text{COND} \rangle \mid \langle \text{CALL} \rangle \mid \text{return } [\langle \text{EXPR} \rangle] \end{array} \right.$$

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Design the rule for the nonterminal `INTERRUPT`, which models an interruption / resumption point. Reuse (and implement) the nonterminal `PARAM_LST` to represent a list of parameters.
- (b) Draw the syntax tree for the interruption point `mark` from the “tic-tac-toe” example (start the tree from the nonterminal `INTERRUPT`).
- (c) Design the rules for the nonterminals `WHILE` and `COND`, considering that the latter has an optional clause “else”.

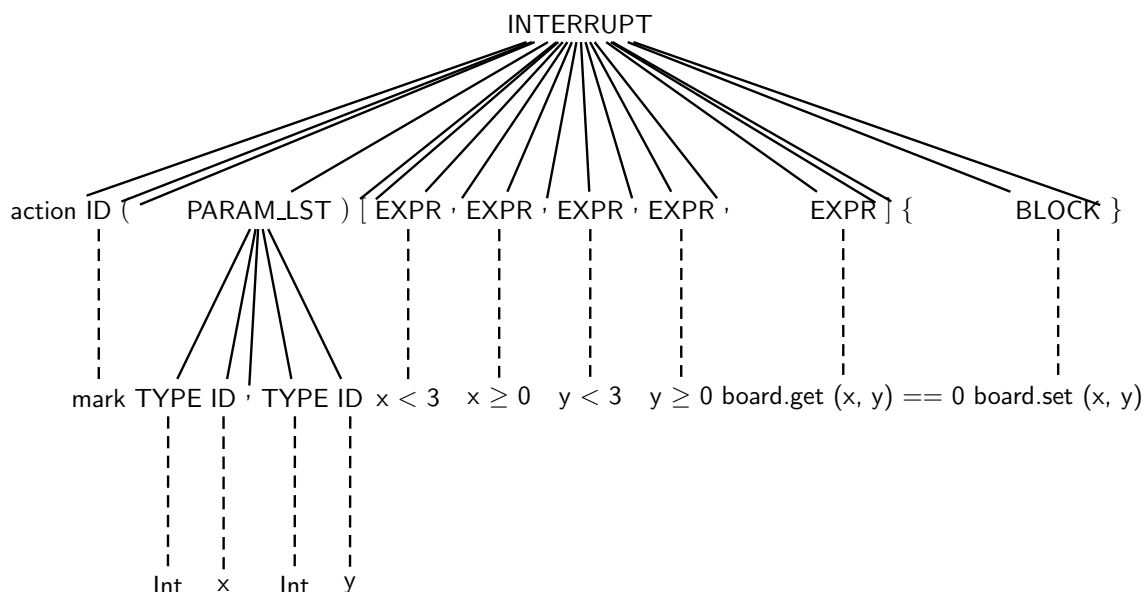
Solution

- (a) From the example and the existing partial grammar, an interruption / resumption point (non-terminal INTERRUPT) is introduced by the keyword `action` and has a name (terminal ID), a possibly empty list of comma-separated formal parameters (nonterminal PARAM_LST), a possibly empty list of comma-separated constraints (each of which is a nonterminal EXPR), and a body consisting of a non-empty list of statements (nonterminal BLOCK). Here is the extension of the grammar, with the rules for nonterminals INTERRUPT and PARAM_LST:

$$\left\{ \begin{array}{l} \langle \text{INTERRUPT} \rangle \rightarrow \text{action ID} \text{ " (" } [\langle \text{PARAM_LST} \rangle] \text{ ") " " [" } \\ \quad [\langle \text{EXPR} \rangle \text{ (" , " } \langle \text{EXPR} \rangle \text{) }^*] \\ \quad \text{ "] " " { " } \langle \text{BLOCK} \rangle \text{ " } \text{ " } \} \text{ " } \\ \hline \langle \text{PARAM_LST} \rangle \rightarrow \langle \text{TYPE} \rangle \text{ ID (" , " } \langle \text{TYPE} \rangle \text{ ID) }^* \end{array} \right.$$

The nonterminals EXPR and TYPE are left unexpanded, as well as the terminal ID. As usual, the square brackets [] indicate the optionality operator. In the grammar rules, round parentheses () and optionality operators [] must not be confused with the same symbols used as terminals, which are enclosed in quotation marks, i.e., “(”, “)”, “[” and “]”.

- (b) Here is the syntax tree of the interruption / resumption point `mark` from the “tic-tac-toe” sample coroutine `play`:



The nonterminals EXPR and TYPE are left unexpanded, as well as the terminal ID. However, for completeness the dashed edges show how these grammar symbols should be eventually expanded for the example, if the appropriate additional grammar rules were provided.

- (c) Again from the example and the existing partial grammar, here is the extension of the grammar, with the simple rules for nonterminals WHILE (*while* loop statement) and COND (*if-then-else* conditional statement):

$$\left\{ \begin{array}{l} \langle \text{WHILE} \rangle \rightarrow \text{while } \langle \text{EXPR} \rangle \text{ " { " } } \langle \text{BLOCK} \rangle \text{ " } \text{ " } \} \text{ " } \\ \hline \langle \text{COND} \rangle \rightarrow \text{if } \langle \text{EXPR} \rangle \text{ " { " } } \langle \text{BLOCK} \rangle \text{ " } \text{ " } [\text{else " { " } } \langle \text{BLOCK} \rangle \text{ " } \text{ " }] \end{array} \right.$$

The nonterminal EXPR is left unexpanded, as well as the terminal ID. To finalize the grammar, besides the rules for TYPE, EXPR and ID, also those for VAR_DCL (variable declaration) and CALL (function call) could be provided (but are not required).

2. Consider a grammar that represents code in a programming language, where spacing conveys the meaning of scope blocks, e.g., Python. The grammar is defined as follows (axiom S):

$$\left\{ \begin{array}{l} 1: S \rightarrow P \\ 2: P \rightarrow L P \\ 3: P \rightarrow \varepsilon \\ 4: L \rightarrow B x n \\ 5: L \rightarrow B y n \\ 6: B \rightarrow b B \\ 7: B \rightarrow \varepsilon \end{array} \right.$$

The terminals b , x , y and n respectively represent:

- b : a blank space
- x : a generic statement (not a block header)
- y : the header of a block of statements
- n : a newline

The nonterminal P models a program, defined as a list of lines (nonterminal L), all of which have a prefix (nonterminal B), which consists of a number of blank spaces (terminal b).

The allowed amount of blank spaces b at the beginning of each line is constrained by the specification of the language as follows. A line that contains the header of a block of statements forces the next line in the program to be indented exactly one blank space b to the right more than the block header. The body of a block consists of all the lines with an indentation level, i.e., number of blank spaces b , greater than the indentation level of the block header. Any statement (except the first after a block header) terminates one or more blocks if it has fewer blank spaces than required. That terminating statement is considered outside of the block(s) terminated. Said differently, the statements that are not the first after a block header, must be indented as much as the previous statement, or even less.

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Design an attribute grammar that checks that the indentation level of each statement is correct, by synthesizing an attribute *valid* that is *true* if the program has a correct indentation, and is *false* otherwise. The following attributes should be employed:

nonterm.	name	type	domain	meaning
S, P	<i>valid</i>	left	boolean	<i>true</i> if the line list has a correct indentation, otherwise <i>false</i>
P	<i>min</i>	right	integer	minimum allowed indentation for the line
P	<i>max</i>	right	integer	maximum allowed indentation for the line
L	<i>is_hdr</i>	left	boolean	<i>true</i> if the line is a block header, <i>false</i> if not
L, B	<i>count</i>	left	integer	amount of indentation

- (b) Compute the values of the attributes for the syntactically and semantically valid string below:

y n b y n b b x n x n

by decorating the syntax tree on the next pages.

Solution

(a) Here is the requested attribute grammar, which uses all and only the proposed attributes:

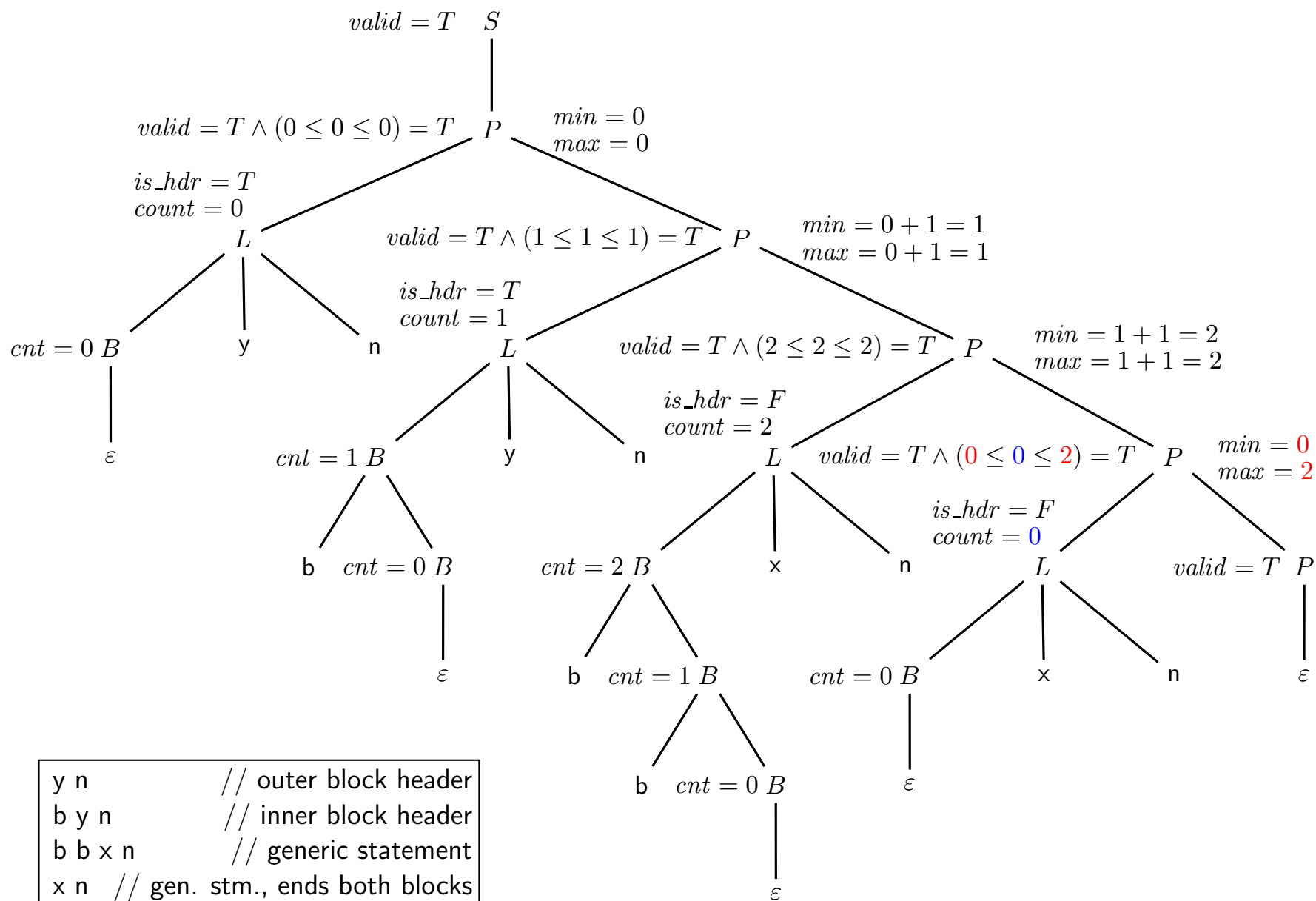
#	<i>syntax</i>	<i>semantics</i>
1:	$S_0 \rightarrow P_1$	$min_1 := 0$ $max_1 := 0$ $valid_0 := valid_1$
2:	$P_0 \rightarrow L_1 P_2$	$min_2 := \text{if } is_hdr_1 \text{ then } count_1 + 1 \text{ else } 0$ $max_2 := \text{if } is_hdr_1 \text{ then } count_1 + 1 \text{ else } count_1$ $valid_0 := valid_2 \wedge (min_0 \leq count_1 \leq max_0)$
3:	$P_0 \rightarrow \varepsilon$	$valid_0 := \text{true}$
4:	$L_0 \rightarrow B_1 x n$	$is_hdr_0 := \text{false}$ $count_0 := count_1$
5:	$L_0 \rightarrow B_1 y n$	$is_hdr_0 := \text{true}$ $count_0 := count_1$
6:	$B_0 \rightarrow b B_1$	$count_0 := count_1 + 1$
7:	$B_0 \rightarrow \varepsilon$	$count_0 := 0$

The synthesized (integer) attribute *count* computes how many blank spaces precede the current statement, i.e., the current indentation level. The synthesized (boolean) statement *is_hdr* distinguishes which statement is a block header rather than a generic (non-header) statement. The core of the attribute grammar is in the rule 2. For each statement, the inherited (integer) attributes *min* and *max* (with $0 \leq min \leq max$) define an interval of positions, which may reduce to one position if $min = max$. A statement can be correctly indented at any position in such an interval, including the extremes *min* and *max*. If the current statement is a block header, both attributes *min* and *max* are set to the current indentation level incremented by one, since the first statement after a block header must be indented to the right exactly one more space. Otherwise, i.e., the current statement is generic (not a block header), attributes *min* and *max* differentiate as follows:

- attribute *min* is set to zero, since the statement might terminate one or more blocks (possibly even all blocks), and in this case its indentation level must be decremented of as many spaces as the number of terminated block(s) (possibly at the beginning of the line)
- attribute *max* is set to the current indentation level, since the next statement may not be indented to the right more than the current one (remember that the indentation level can grow only after a block header)

The synthesized (boolean) attribute *valid* checks that the subtree is itself correct and that the indentation level of the current statement satisfies both bounds *min* and *max*, i.e., falls in the interval. See also the decorated tree of point (b), with the explicit computation of *min*, *max* and *valid* (for the simpler attributes *count* and *is_hdr* only the result is shown).

Therefore, the attribute grammar is reasonably correct; see also point (b) for an example. Furthermore, the dependency graph is acyclic, thus the grammar is computable.



(b) Here is the tree of string `y n b y n b b x n x n` decorated with attribute values:

4 Bonus question (5 points)

Consider the BNF grammar G below, over the two-letter terminal alphabet $\Sigma = \{ a, b \}$ (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow a S b S \\ S \rightarrow a S \\ S \rightarrow \varepsilon \end{array} \right.$$

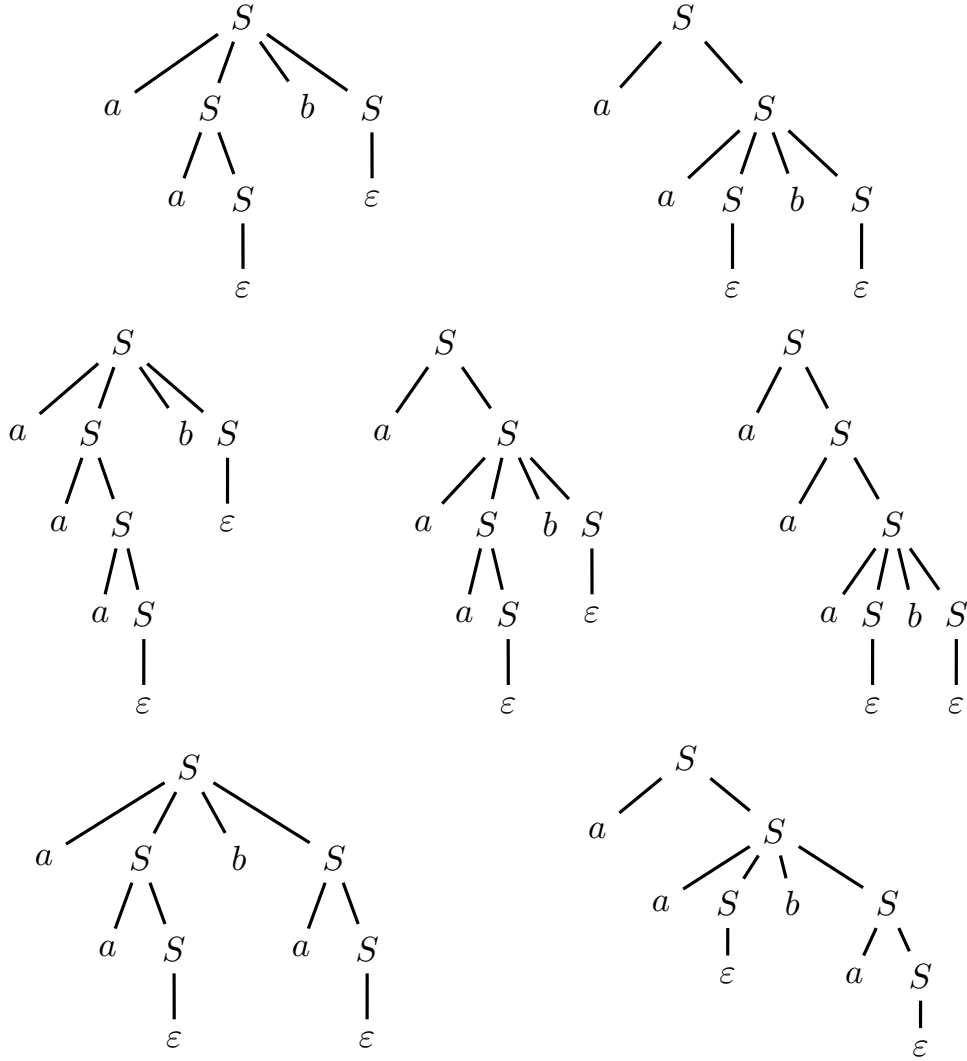
Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) List all the strings of length ≤ 3 (less than or equal to three) that belong to language $L(G)$.
 - (b) Argue that grammar G is ambiguous by exhibiting the two shortest ambiguous strings generated by G , and all their syntax trees.
 - (c) Find a non-ambiguous BNF grammar G' equivalent to grammar G , and justify your answer.
 - (d) Is language $L(G)$ deterministic ELR or ELL ? Explain your answer.
-

Solution

- (a) Here are the valid short strings of length ≤ 3 (seven in total – in lexicographic order): ε , a , aa , ab , aaa , aab and aba .
- (b) There are in fact three shortest ambiguous strings (in lexicographic order): aab of length 3, and $aaab$ and $abab$, both of length 4.

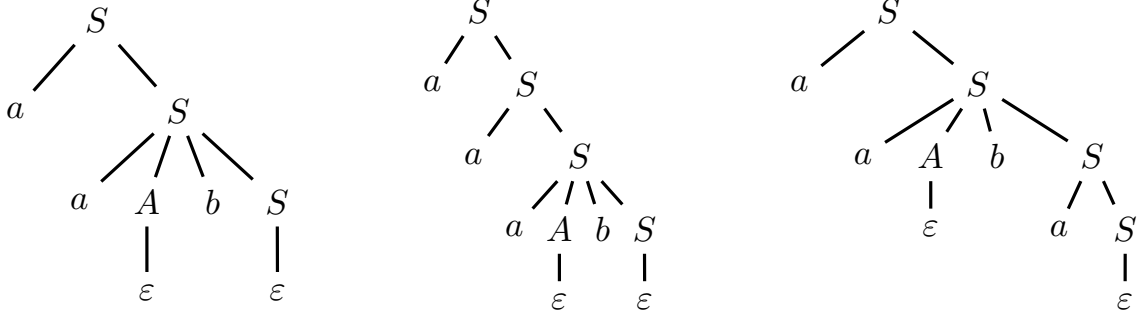
Here are all their syntax trees (seven in total – two for aab , three for $aaab$, and two for $abab$):



- (c) Basically, grammar G generates the Dyck language over the alphabet Σ (see rule $S \rightarrow aSbS$), where there may be an excess of letters a over letters b (see rule $S \rightarrow aS$). Ambiguity stems from the possibility of generating an excess of letters a in different parts of a tree, as the trees listed at point (b) demonstrate. A possible non-ambiguous BNF grammar G' is (axiom S):

$$G' \left\{ \begin{array}{ll} S \rightarrow aS & // \text{Dyck rules with exceeding letters } a \text{ before each group} \\ S \rightarrow aASbS \\ S \rightarrow \varepsilon \\ \hline A \rightarrow aAbA & // \text{Dyck rules without exceeding letters } a \\ A \rightarrow \varepsilon \end{array} \right.$$

First, up to a length ≤ 3 grammar G' generates all and only the seven short strings of point (a). Second, by structure grammar G' generates a Dyck language, such that any prefix of each top-level Dyck language group (and hence the whole string itself) may contain more letters a than letters b . Therefore, grammar G' is reasonably equivalent to grammar G . Furthermore, the three strings of point (b) are no longer ambiguous for grammar G' , since now each of them has exactly one tree:



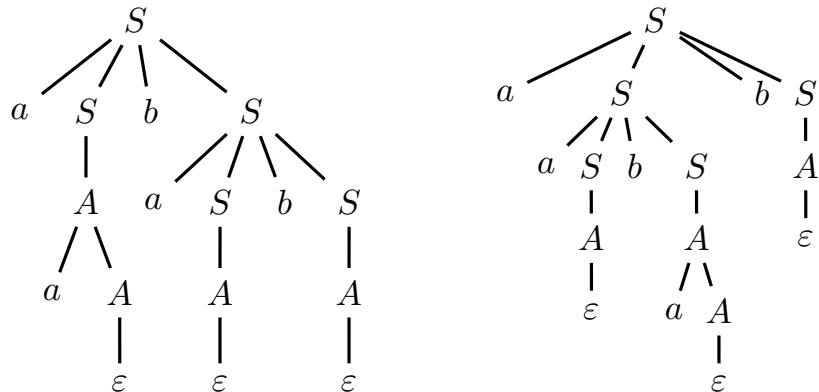
In fact, nonterminal S imposes that the exceeding letters a , if any, are generated before the first letter a that matches a letter b . Therefore grammar G' is reasonably non-ambiguous. This observation suggests a variant non-ambiguous EBNF form for G' , which consists of a list of instances of the Dyck language, initiated, separated and terminated by a^* (axiom S):

$$G'_{\text{EBNF}} \begin{cases} S \rightarrow a^* (a A b a^*)^* & // \text{ list of Dyck language instances} \\ A \rightarrow (a A b)^* & // \text{ Dyck rule in EBNF form} \end{cases}$$

Formulating a non-ambiguous grammar for this language is surprisingly difficult. The following grammar G'' , for example, while it appears non-ambiguous for strings up to length 4, still has ambiguities for strings of length 5 such as $aabab$:

$$G'' \begin{cases} S \rightarrow a S b S & // \text{ Dyck rule} \\ S \rightarrow A \\ \hline A \rightarrow a A & // \text{ exceeding letters } a \text{ at the deepest level} \\ A \rightarrow \varepsilon \end{cases}$$

The ambiguity arises from the fact that while forcing an order on the derivations is intended to make the exceeding letters a appear at the deepest nesting level of a Dyck group, the siblings of that group may contain other sub-groups that are deeper still. The two syntactic trees for the string $aabab$ according to grammar G'' demonstrate this observation:



In the first tree the structure of the string (replacing matched a and b s with parenthesis for clarity) is interpreted as $(a)()$, while in the second tree the structure is $(()a)$.

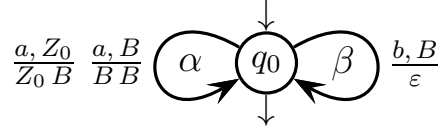
- (d) Grammars G' and G'_{EBNF} are ELR(1), which can be (relatively) easily proven by deriving their pilot graphs.

Alternatively, an easy-to-find deterministic pushdown automaton can correctly recognize a string of the language in this way:

- moves of the pushdown automaton:
 - the automaton pushes a symbol B whenever it reads a character a
 - the automaton pops a symbol B whenever it reads a character b
- conditions of acceptance and rejection:
 - if the current character is a letter b and the stack is empty, the automaton halts and rejects
 - otherwise the automaton continues and eventually accepts the whole string

So in the end all and only the strings with at least as many letters a as letters b are accepted, which is precisely the language $L(G)$. It is well known that the family of ELR(1) languages coincides with that of the deterministic languages (with recognition by final state), therefore also language $L(G)$ is ELR(1). This rapidly concludes the argument.

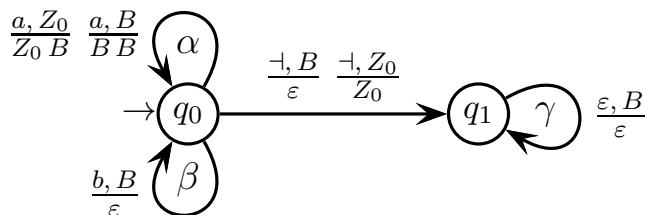
For completeness, here is an implementation of such a deterministic pushdown automaton, which is not a syntax analyzer (as it is not related to a grammar). It has only one state, which is both initial and final, and recognizes by final state (even if the stack is not empty):



Note that the automaton accepts the whole valid string and any prefix thereof. In fact the automaton repeatedly reads letters a and b (starting with a) on the self-loops α and β , respectively, constantly recognizes the input prefix scanned so far (since its only state q_0 is final) and goes on reading the rest of the input (if any), but if it reaches a position in the input where the number of letters b is greater than that of letters a , then the stack top is Z_0 with current input b , and as there is no such move, it halts and rejects. Thus as long as the input is valid, it is accepted, the analysis continues and may even reach the input end if the whole string is valid, otherwise the input is immediately rejected and the analysis breaks. In practice the stack acts as an integer counter, which is incremented and decremented by the letters a and b , respectively.

The fact that the above pushdown automaton accepts any prefix of a valid string is not surprising. In fact, language $L(G)$ has the property that any prefix of a valid string is itself a valid string. For instance, the valid string $abab$ listed last at point (a) has three prefixes, namely ϵ , a and ab , and such prefixes are themselves valid strings, see again point (a). Another example is the pair of ambiguous strings $abab$ and $ababa$, where the first string is a prefix of the second.

The same deterministic pushdown automaton, in a version that recognizes (still deterministically) the whole input string by empty stack, is shown below:



This version has two states, i.e., q_0 (initial) and q_1 , but neither is final. To accept only the whole valid input string (not also prefixes thereof), we assume that the input is ended by a terminator “ $\dashv$ ” (same as for syntax analyzers). The acceptance condition is that the terminator has been read and the stack is empty. This happens only when the automaton switches to state q_1 , where it possibly flushes the stack (unless the stack was already empty) on the self-loop γ without reading the input any more, and so accepts as soon as the stack gets empty. As for the rest, this empty-stack version works as the previous final-state version, and uses the stack as an integer counter.

Finally, grammars G' and G'_{EBNF} are *not* $\text{ELL}(k)$ for any k , which makes them of no use in determining if the language is ELL. However we can observe that language $L(G)$ contains strings of type $a^m b^n$ with $m \geq n \geq 0$ (or $a^* a^n b^n$ with $n \geq 0$), which constitute a well-known non- $\text{ELL}(k)$ sublanguage, for any $k \geq 1$. In fact, these strings are all valid for the language. This makes $L(G)$ unlikely to be ELL.